BUILDEVCON · MARIADB / MYSQL ON KUBERNETES

Run and Operate MariaDB in a Cloud Native Way

Using the MariaDB Operator

Martin Montes · Lead Engineer, MariaDB Corporation

Martin Montes

Lead Engineer

MariaDB Corporation

mmontes11

mmontesperez

@m_montes11

mmontes11

whoami

Creator & Lead Maintainer

mariadb-operator — Kubernetes operator for MariaDB, built with Go and controller-runtime.

Specialized in **Kubernetes · Golang · Databases**

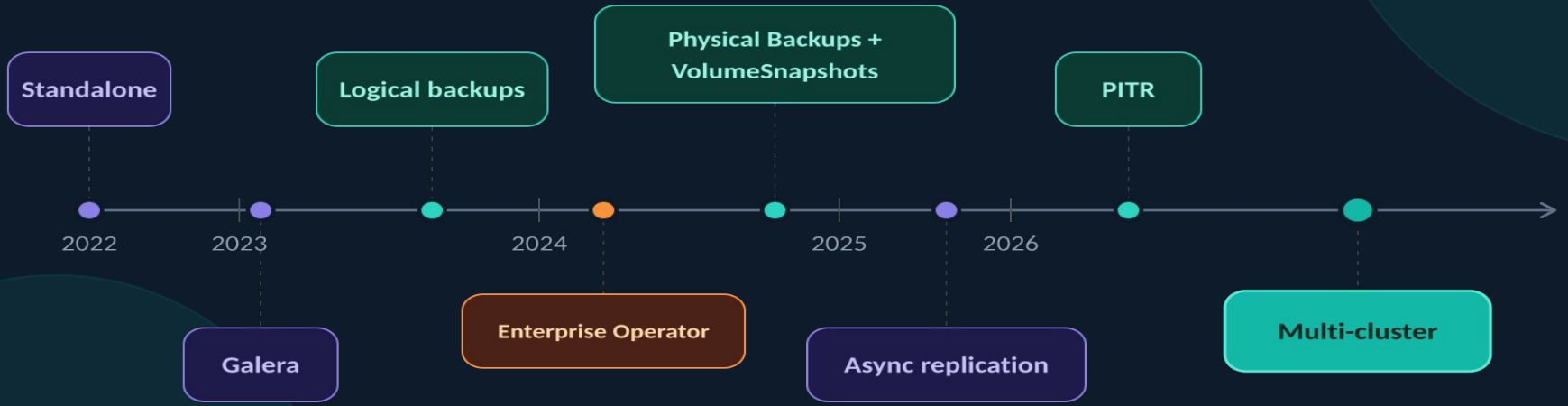
Maintains the operator, ships features, grows the community.

Cloud-native | **Open Source** | **Cloud-agnostic**

“Run and operate MariaDB in a cloud native way — declaratively manage your database using Kubernetes CRDs rather than imperative commands.”

Project Timeline

From standalone MariaDB to full multi-cluster replication — in 4 years



Community & Adoption

mariadb-operator · github.com/mariadb-operator/mariadb-operator

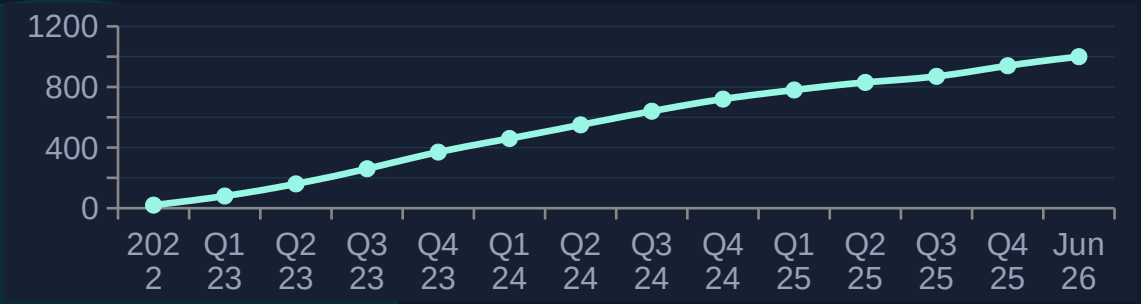
~1K
GitHub Stars

80+
Contributors

4K+
Commits

8M+
Image Pulls

★ GitHub Stars over time



157+
Releases

v26.06
Latest

2022
Since

DAY 1

Cluster Setup

Provision a production-ready MariaDB cluster — declaratively.

DAY 1 · 1 / 6

Topology

Choose your HA strategy

- **Standalone:** single replica — dev / test only
- **Replication:** semi-sync, automated failover — recommended for production
- **Galera:** synchronous multi-master — alternative HA topology
- **Multi-cluster:** cross-cluster replication for multi-region deployments and blue-green upgrades
- All topologies share the same MariaDB CRD — just flip feature flags

```yaml
# Standalone — single node (dev, test)
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  replicas: 1
---
# Replication — semi-sync, auto failover
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  replicas: 3
  replication:
    enabled: true
    semiSyncEnabled: true
    primary:
      autoFailover: true
---
# Galera — synchronous multi-master
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  replicas: 3
  galera:
    enabled: true
---
# Multi-cluster — cross-cluster replication
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  replicas: 3
  replication:
    enabled: true
  multiCluster:
    enabled: true
    primary: mariadb-eu-south
    members:
      - name: mariadb-eu-south
      - name: mariadb-eu-central
```

DAY 1 · 2 / 6

Storage

Persistent volumes for your data

- **StorageClass:** defaults to cluster default — you can bring your own
- **Local storage:** strongly recommended — topolvm or local-path-provisioner avoids network I/O latency
- **Block storage:** high IOPs preferred when local isn't an option — e.g. AWS io2
- **Volume resize:** update size at runtime, operator handles the expansion
- StorageClass must have allowVolumeExpansion: true for online resize

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  storage:
    # Choose any StorageClass in your cluster.
    # Local storage (e.g. topolvm) avoids
    # network latency.
    storageClassName: topolvm
    size: 100Gi

    # Grow the volume at runtime: bump size and
    # the operator handles the resize.
    # StorageClass must have:
    #   allowVolumeExpansion: true
    resizeInUseVolumes: true
    waitForVolumeResize: true
```

DAY 1 · 3 / 6

Compute

Right-size CPU & memory for MariaDB

- **Requests = Limits:** set both identical — Kubernetes assigns Guaranteed QoS
- **innodb_buffer_pool_size:** set to 70–80% of memory limit for best performance
- Guaranteed QoS pods are never evicted under node pressure
- myCnf inlined in the CR — operator mounts it automatically

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  resources:
    requests:
      cpu: "4"
      memory: 8Gi
    limits:
      cpu: "4"
      memory: 8Gi

  # innodb_buffer_pool_size should be
  # 70-80% of the memory limit.
  myCnf: |
    [mariadb]
    innodb_buffer_pool_size=6400M
```

DAY 1 · 4 / 6

Scheduling

Isolate and protect your database pods

- **Anti-affinity:** antiAffinityEnabled spreads replicas across nodes
- **Node selector:** select the right instance type (high IOPs, 10Gbps+) and pin pods to dedicated DB nodes
- **Topology spread:** distribute evenly across availability zones
- **Tolerations:** combine with node taints to dedicate nodes exclusively to databases
- **priorityClassName:** system-node-critical — prevents eviction under node pressure

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  replicas: 3
  # Spread each replica on a different node.
  affinity:
    antiAffinityEnabled: true
  # Pin Pods to DB-dedicated nodes.
  nodeSelector:
    node.kubernetes.io/instance-type: m6in.xlarge
  # Spread across availability zones.
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
  # Tolerate the taint on dedicated DB nodes.
  tolerations:
    - key: "mariadb.com/dedicated"
      operator: "Exists"
      effect: "NoSchedule"
  # Limit voluntary disruptions.
  podDisruptionBudget:
    maxUnavailable: "1"
  # Prevent eviction under node pressure.
  priorityClassName: system-node-critical
```

DAY 1 · 5 / 6

TLS

Encrypted connections by default

- **tls.enabled:** operator auto-issues CA + server + client certs
- **tls.required:** reject all non-TLS connections for production
- **cert-manager:** delegate issuance to a ClusterIssuer if preferred
- **Bring your own CA:** provide the CA keypair in a Secret
- Auto-renewal — operator rotates certs before expiry without downtime

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  tls:
    enabled: true
    # Reject any non-TLS connection attempt.
    required: true

    # Option A (default): operator issues and
    # rotates certificates automatically.

    # Option B: delegate issuance to cert-manager.
    serverCertIssuerRef:
      name: root-ca
      kind: ClusterIssuer
    clientCertIssuerRef:
      name: root-ca
      kind: ClusterIssuer

    # Option C: bring your own CA stored
    # in a Kubernetes Secret.
    serverCASecretRef:
      name: mariadb-server-ca
    clientCASecretRef:
      name: mariadb-client-ca
```

DAY 1 · 6 / 6

Metrics

Prometheus-native observability

- **metrics.enabled:** single flag — operator provisions everything
- **Multi-target exporter:** one Deployment scrapes all replicas, no sidecars
- **ServiceMonitor:** auto-created for prometheus-operator integration
- **Credentials:** monitoring user + password managed by the operator
- **scrapeTimeout:** tune interval and timeout to match your Prometheus setup

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  metrics:
    enabled: true
    # Operator provisions a multi-target
    # Prometheus exporter — no sidecars.
    exporter:
      resources:
        requests:
          cpu: 50m
          memory: 64Mi
        limits:
          memory: 512Mi
    # Operator manages credentials automatically.
    username: monitoring
    passwordSecretKeyRef:
      name: mariadb
      key: password
    # Auto-registers metrics in Prometheus.
    serviceMonitor:
      interval: 10s
      scrapeTimeout: 10s
```

DAY 2

Operations

Backup, recovery, updates, and maintenance — all declarative.

DAY 2 · 1 / 6

Physical Backups

Fast, consistent full backups

- **mariadb-backup:** hot backup — zero downtime, taken from a replica
- **VolumeSnapshots:** Kubernetes-native, near-instant, CSI-driver-based
- **Storage:** S3-compatible, Azure Blob, PVC, or Kubernetes volumes
- **Scheduling:** cron expression or on-demand trigger

- **onDemand:** update the identifier field to schedule an immediate backup at any time
- **Retention:** maxRetention auto-cleans old backups (e.g. 720h = 30 days)

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: PhysicalBackup
metadata:
  name: physicalbackup
spec:
  mariaDbRef:
    name: mariadb
  schedule:
    cron: "0 2 * * *"      # daily at 02:00
    # Update identifier to trigger on-demand:
    onDemand: "1"
  target: Replica          # zero impact on primary
  compression: bzip2
  maxRetention: 720h       # 30 days
  storage:
    # S3-compatible (AWS S3, MinIO, GCS, …)
    s3:
      bucket: physicalbackups
      prefix: mariadb
      endpoint: s3.amazonaws.com
      region: us-east-1
      accessKeyIdSecretKeyRef:
        name: s3-credentials
        key: access-key-id
      secretAccessKeySecretKeyRef:
        name: s3-credentials
        key: secret-access-key
    # azureBlob: { … }      # Azure Blob Storage
    # volumeSnapshot: { … } # consistent, near-instant
```

DAY 2 · 2 / 6

Point-In-Time Recovery

Restore to any second in history

- **Binary log archival:** agent sidecar continuously streams binlogs to S3
- **Base backup + binlogs:** PITR = restore physical backup then replay logs
- **targetRecoveryTime:** RFC3339 timestamp — operator finds the right backup
- **Last recoverable time:** visible in PointInTimeRecovery status — drives RPO
- MariaDB 10.8+ required · Replication topology

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: PointInTimeRecovery
metadata:
  name: pitr
spec:
  physicalBackupRef:
    name: physicalbackup
  archiveTimeout: 5m
  storage:
    s3:
      bucket: binlogs
      endpoint: s3.amazonaws.com
      accessKeyIdSecretKeyRef:
        name: s3-credentials
        key: access-key-id
      secretAccessKeySecretKeyRef:
        name: s3-credentials
        key: secret-access-key
---
# Restore to a specific point in time:
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  bootstrapFrom:
    pointInTimeRecoveryRef:
      name: pitr
    targetRecoveryTime: "2025-06-18T14:37:52Z"
```

DAY 2 · 3 / 6

Restore & Bootstrap

Spin up new clusters from existing data

- **bootstrapFrom:** declare the restore source directly in the MariaDB CR
- **PhysicalBackup ref:** restore from the closest backup before targetRecoveryTime
- **PITR:** combine physical backup + binlog replay for point-in-time restoration
- Operator pauses failover operations while restore is in progress
- Staging storage PVC avoids exhausting node disk for large backups

```yaml
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  replicas: 3
  replication:
    enabled: true

  bootstrapFrom:
    # A) Restore closest PhysicalBackup.
    backupRef:
      name: physicalbackup
      kind: PhysicalBackup

    # B) PITR — restore base backup then
    # replay binlogs up to targetRecoveryTime.
    pointInTimeRecoveryRef:
      name: pitr
    targetRecoveryTime: "2025-06-18T14:37:52Z"

    stagingStorage:
      persistentVolumeClaim:
        resources:
          requests:
            storage: 100Gi
        accessModes:
          ReadWriteOnce
```

DAY 2 · 4 / 6

Updates

Zero-downtime rolling upgrades

- **ReplicasFirstPrimaryLast:** roll replicas one-by-one, primary last — default
- **Never:** freeze updates — useful during maintenance windows or staged rollouts
- **OnDelete:** full manual control — delete a pod to trigger its update
- **autoUpdateDataPlane:** automatically updates operator init container and sidecar images to match the operator version

```yaml
# Default rolling strategy: replicas first,
# then the primary. Minimises disruption.
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  updateStrategy:
    type: ReplicasFirstPrimaryLast
    autoUpdateDataPlane: true
---
# Pause all automatic image updates during
# a maintenance freeze or staged rollout.
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  updateStrategy:
    type: Never
---
# Full manual control: delete a Pod to
# trigger its individual update.
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  updateStrategy:
    type: OnDelete
```

DAY 2 · 5 / 6

Configuration

Live my.cnf changes without manual restarts

- **myCnf inline:** changes trigger a rolling update automatically
- **ConfigMap ref:** label it k8s.mariadb.com/watch — operator detects edits
- **Typical tuning:** innodb_buffer_pool_size, max_connections, binlog settings
- GitOps-friendly: commit the ConfigMap, CI applies it, operator reloads

```yaml
# Label the ConfigMap with
# k8s.mariadb.com/watch — any edit
# triggers a rolling update.
apiVersion: v1
kind: ConfigMap
metadata:
  name: mariadb-config
  labels:
    k8s.mariadb.com/watch: ""
data:
  my.cnf: |
    [mariadb]
    innodb_buffer_pool_size=9600M
---
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
metadata:
  name: mariadb
spec:
  resources:
    requests:
      memory: 12Gi
    limits:
      memory: 12Gi
  myCnfConfigMapKeyRef:
    name: mariadb-config
    key: my.cnf
```

DAY 2 · 6 / 6

Maintenance

Safe operational windows with fine-grained control

- **Suspend:** pause all operator reconciliation for fully manual control
- **Cordon:** remove pods from Service endpoints — blocks new connections
- **Drain:** gracefully terminate long-running queries after a grace period
- **ReadOnly:** prevent writes — ideal before a cluster switchover
- Combine cordon + drain + readOnly for a safe zero-data-loss maintenance window

```yaml
# Suspend: pause all operator reconciliation.
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  suspend: true
---
# Maintenance mode: fine-grained control
# over traffic during planned windows.
# Ideal before a multi-cluster switchover.
apiVersion: k8s.mariadb.com/v1alpha1
kind: MariaDB
spec:
  maintenance:
    # Block new connections at Service level.
    enabled: true
    cordon: true
    # Gracefully terminate long-running queries.
    drainConnections: true
    drainGracePeriodSeconds: 30
    # Prevent accidental writes.
    readOnly: true
```

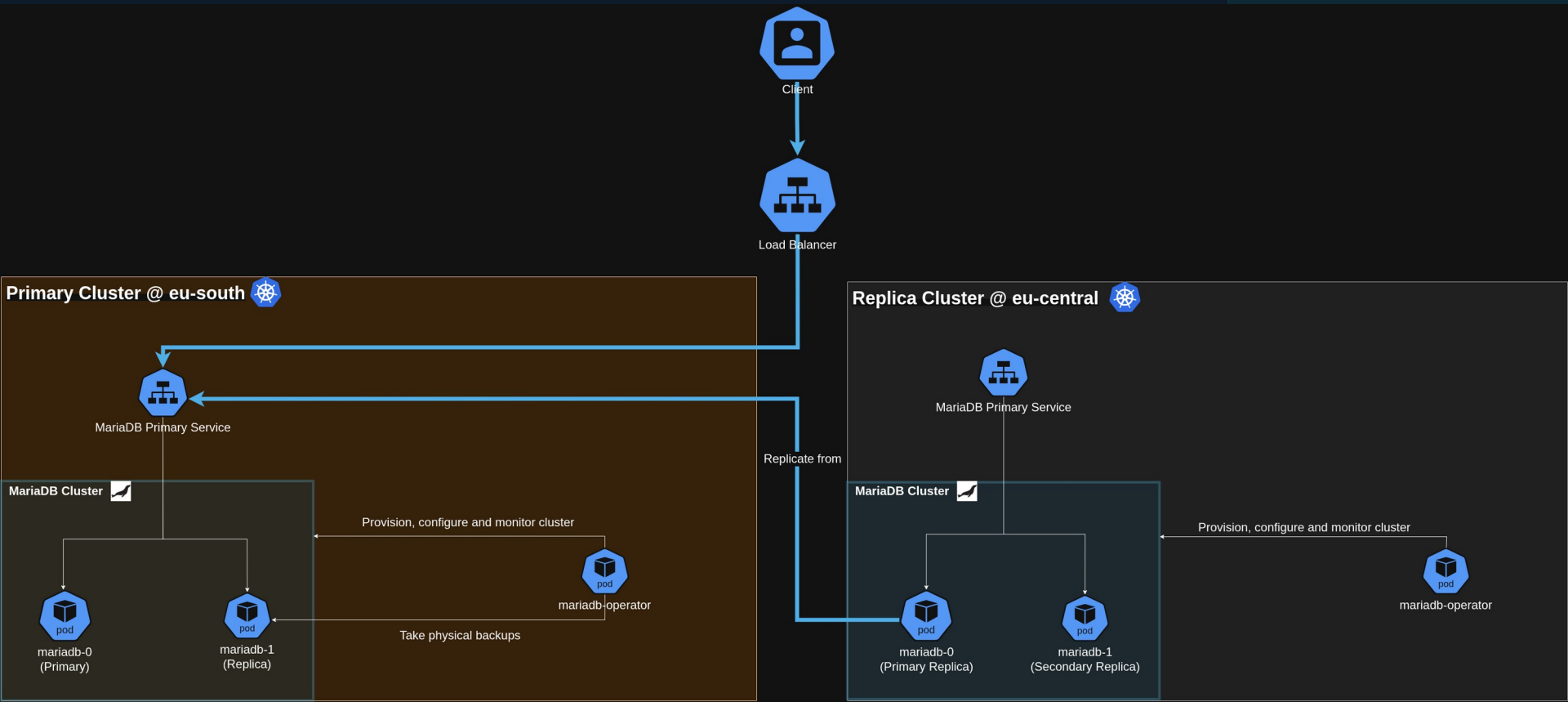
DEMO

Multi-Cluster

Cross-cluster replication · Multi-region HA · Blue-green upgrades

Architecture

Cross-region deployment using the multi-cluster topology



Thank you!

Run and Operate MariaDB in a Cloud Native Way

Useful links

Documentation	`github.com/mariadb-operator/mariadb-operator/blob/demos-multi-cluster/docs/README.md`
Demo	`github.com/mariadb-operator/mariadb-operator/tree/demos-multi-cluster/demos/multi-cluster`
GitHub repo	`github.com/mariadb-operator/mariadb-operator`

Scan to open repo

Martin Montes · Lead Engineer, MariaDB Corporation